

ONLINE EXAM MONITORING & INTEGRITY ANALYTICS PLATFORM

Project Documentation

A web-based examination monitoring system using browser activity monitoring, OpenCV Haar Cascade face detection, integrity scoring, data analytics and AI-generated reports.

Prepared by

Rasika Vinod Dhumane

2026

Abstract

The Online Exam Monitoring & Integrity Analytics Platform is designed to support online examinations by recording selected browser and webcam-related events during an active examination session. The system combines a Flask web application, SQLite database, OpenCV Haar Cascade face detection, rule-based integrity scoring, Pandas-based analytics, K-Means session clustering, and a LangChain-based AI report agent.

The platform follows a controlled examination workflow: candidate registration/login, identity photo capture, dashboard access, system readiness check, declaration, examination, monitoring, submission, integrity calculation, and post-exam analytics. The system records events such as tab switches, focus loss, and changes in detected face status. These records are used to produce an integrity score and risk label for the completed session.

This document describes the system architecture, modules, database design, monitoring approach, scoring method, analytics, AI report generation, user interface, testing approach, limitations, and future scope.

Table of Contents

Section	Description
1. Introduction	Background, problem statement and objectives
2. Requirements & Technology Stack	Functional requirements, tools and libraries
3. System Architecture & Workflow	Architecture, data flow and examination sequence
4. System Modules	Major application components
5. Database Design & Security	SQLite schema, relationships and basic protection
6. Monitoring & Event Detection	Haar Cascade and browser-event monitoring
7. Integrity Scoring & Analytics	Score calculation, risk labels and K-Means
8. AI Report Generation & Validation	LangChain/Gemini report generation and validation
9. Testing, Limitations & Future Scope	Testing strategy, limitations and improvements
Conclusion & References	Final summary and technical references

1. Introduction

1.1 Background

Online examinations provide flexibility and remote access, but they also require mechanisms for recording examination behaviour. A monitoring system can help institutions review events that occur during an examination instead of depending only on the candidate's submitted answers.

1.2 Problem Statement

A conventional online test interface may record answers and submission status without maintaining a structured integrity record. Important behavioural events, such as leaving the examination window or temporarily disappearing from the webcam view, may therefore be difficult to review after submission.

1.3 Proposed Solution

The proposed platform maintains an exam session and stores monitoring events against that session. During the exam, the webcam frame is processed with OpenCV Haar Cascade to detect whether a face is present, absent, or multiple faces are visible. Browser-side events are sent to the Flask backend. At submission, the stored session data is converted into an integrity score, risk label and report.

1.4 Objectives

- Provide candidate registration and secure login with password hashing.
- Capture or upload a candidate identity photograph and validate it with face detection.
- Perform a pre-exam browser/camera readiness check before examination.
- Monitor face presence and selected browser activity during an active exam.
- Store time-stamped monitoring events in SQLite for later review.
- Calculate a transparent integrity score using event penalties and face-presence duration.
- Provide analytical views including event frequency, score trends and session clustering.
- Generate a concise AI-assisted integrity report from supplied session data and validate its consistency.

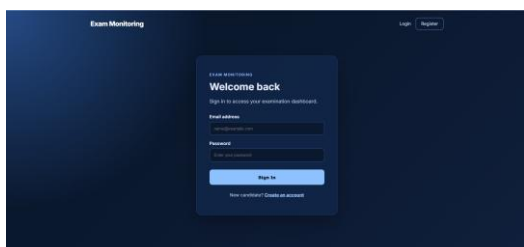


Figure 1. Candidate login screen

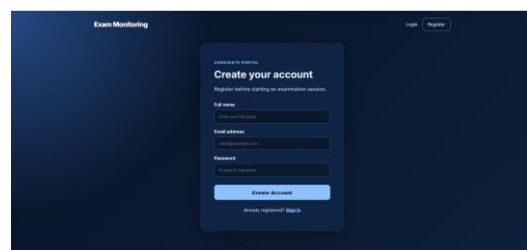


Figure 2. Candidate registration screen

2. Requirements and Technology Stack

2.1 Functional Requirements

Requirement	System Behaviour
Candidate account	Register and authenticate candidates using email and password.
Identity photo	Capture webcam photo or upload an image before dashboard access.
System readiness	Check browser support, camera permission and camera accessibility.
Exam session	Create an active exam session and maintain its start/submission times.
Face monitoring	Process webcam frames using OpenCV Haar Cascade.
Browser monitoring	Record tab-switch and focus-loss events from the examination page.
Integrity scoring	Calculate score, penalties, face-presence ratio and risk label.
Analytics	Display event counts and integrity-score information after submission.
AI report	Generate a factual integrity summary from calculated session data.

2.2 Technology Stack

Layer	Technology	Purpose
Web backend	Python + Flask	Routes, authentication, session workflow and APIs
Database	SQLite	Candidates, exam sessions, events and integrity reports
Computer vision	OpenCV Haar Cascade	Face-presence detection
Data processing	Pandas	Session-level data preparation and score calculation
Visual analytics	Matplotlib + Seaborn	Analytical plots and exploratory visualizations
Machine learning	Scikit-learn K-Means	Session behavioural clustering
AI integration	LangChain + Gemini	AI-assisted integrity report generation
Frontend	HTML, CSS, JavaScript	Candidate pages, exam UI and browser monitoring
Security support	Werkzeug	Password hashing and secure file utilities

2.3 Development Environment

The project is implemented as a Python web application with a local SQLite database. The requirements file includes Flask, OpenCV, NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, python-dotenv, LangChain components, Streamlit and pytest.

3. System Architecture and Workflow

The architecture separates the candidate-facing web interface, Flask application logic, monitoring layer, persistent storage, scoring/analytics layer and AI reporting layer. The browser communicates with Flask through normal web requests and monitoring APIs. Flask stores session events in SQLite and invokes the scoring and report components after submission.

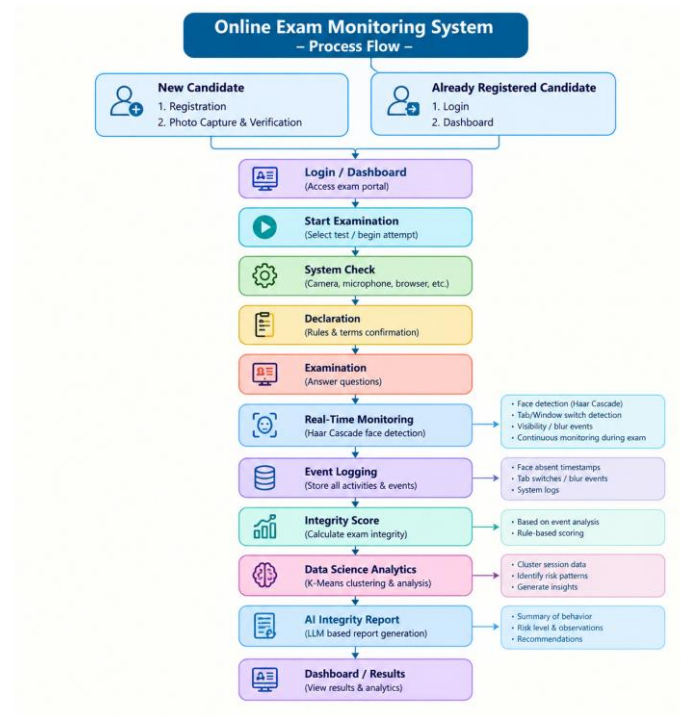


Figure 3. Proposed architecture and data flow

3.1 Examination Workflow

For a new candidate, the flow is Registration → Identity Photo Capture → Dashboard. For every examination attempt, the flow is Dashboard → Start Exam → System Check → Declaration → Start Examination → Existing Exam Interface → Submit Exam → Integrity Report/Analytics.

Stage	Main Action
1. Authentication	Candidate signs in and receives a server-side session.
2. Identity verification	Photo is captured/uploaded and checked for exactly one detected face.
3. System check	Browser support and camera permission/accessibility are checked.
4. Declaration	Candidate accepts the examination declaration.
5. Examination	Question interface starts and monitoring APIs become active.
6. Monitoring	Face status and browser events are recorded with timestamps.
7. Submission	Exam session is marked submitted.
8. Post-processing	Open face-absence intervals are closed, score is calculated and AI report is generated.
9. Analytics	Candidate can view the completed session's score, events and charts.

4. System Modules

The platform's core modules cover the full candidate journey. The Flask application handles registration and login, storing each candidate's email, full name, hashed password and identity-photo filename, and redirects candidates with no saved photo to the photo-capture step. Identity verification is done through a webcam capture or upload page, where the Haar Cascade detector accepts the photo only when exactly one face is present, preventing an empty or multi-person registration image from being stored. Before the exam begins, a system check confirms browser compatibility and camera access; this check is part of the Start Exam flow rather than a permanent sidebar destination, since face-presence monitoring itself is performed during the live exam rather than as a readiness test. Once the check passes, the candidate reaches the declaration page and explicitly starts the examination.

5. Database Design and Security

5.1 SQLite Database

The application uses SQLite through Python's `sqlite3` module. Foreign-key enforcement is enabled for the connection. The database contains separate tables for candidates, examination sessions, monitoring events and generated integrity reports.

Table	Important Fields	Purpose
candidates	id, full_name, email, password, photo_filename	Stores candidate account and identity-photo information.
exam_sessions	id, candidate_id, exam_name, status, started_at, submitted_at	Represents each examination attempt.
session_events	id, session_id, event_type, event_timestamp, details, duration_seconds	Stores monitoring and browser events.
session_integrity_reports	session_id, integrity_score, risk_label, face_presence_ratio, penalties, report_text	Stores completed-session scoring and AI report data.

5.2 Relationships

One candidate can have multiple examination sessions. Each examination session can contain multiple monitoring events. A completed session has one corresponding integrity report. The foreign-key fields connect sessions to candidates, events to sessions, and reports to sessions.

5.3 Security Measures

- Passwords are stored using Werkzeug password hashing rather than plain text.
- Candidate ownership is checked before accessing a session or its analytics.
- Exam monitoring APIs require an authenticated candidate and an active session.
- Uploaded images are restricted to supported image extensions and a configured size limit.
- API keys for the AI component are loaded from an environment variable rather than written directly into the report prompt.

For deployment, the Flask secret key should be replaced with a strong environment-based secret, and production configuration should disable debug mode.

6. Monitoring and Event Detection

6.1 Haar Cascade Face Detection

The project uses OpenCV's Haar Cascade classifier for face detection. A webcam frame is decoded from the browser, converted to grayscale, and passed to the Haar Cascade detector. The detector returns face bounding boxes. The application then classifies the current frame as Face Detected, No Face Detected, or Multiple Faces Detected.

The implementation does not perform face recognition or identity matching. It is used for face-presence monitoring only. When the face becomes absent, the system records the beginning of the absence interval. When the face returns, the interval is completed and its duration is stored.

6.2 Browser Activity Monitoring

The examination page can report selected browser activity events to the backend. The current event API accepts tab-switch, window-focus-loss and window-focus-returned events. Events are stored with timestamps so that the completed session can be reviewed.

6.3 Rule-Based Suspicious Event Detection

The application also defines configurable thresholds for selected events. The current rule configuration uses more than 3 tab switches, more than 3 focus losses, or more than 60 seconds of face absence as threshold conditions for suspicious-event reporting. These rules are separate from the final numerical integrity score.

7. Integrity Scoring and Data Analytics

7.1 Integrity Score

The scoring module normalizes historical and live event names and calculates an event penalty. The current weights are: tab_switch = 5 points, window_focus_lost = 3 points, and no_face = 5 points per event. Face absence duration is also used to calculate a face-presence ratio.

Face presence ratio = $1 - (\text{total face-absence duration} / \text{total examination duration})$. The ratio is constrained to the 0–1 range. A presence penalty of $(1 - \text{face presence ratio}) \times 30$ is then applied. The final integrity score is $100 - \text{event penalty} - \text{presence penalty}$ and is constrained to 0–100.

7.2 Risk Labels

Integrity Score	Risk Label
80–100	Low
50–79.99	Medium
0–49.99	High

7.3 Data Science Analytics

The analytics module aggregates session-level features: tab-switch count, focus-loss count, face-absence count, face-absence duration and integrity score. These features are standardized with StandardScaler and processed using K-Means with three clusters. Cluster names are mapped using the average integrity score so that the numerical cluster identifiers become readable behavioural risk groups.

The project also creates an integrity-score distribution, suspicious-event frequency chart, K-Means session-risk scatter plot and event-frequency heatmap. These visualizations support post-exam analysis rather than changing the underlying event records.



Figure 5. Session analytics visualization

8. AI Integrity Report Generation and Validation

8.1 AI Report Agent

The AI report component uses LangChain's ChatPrompt Template together with the Google Gemini integration. The application loads GEMINI_API_KEY from the environment, prepares a structured prompt, and supplies the calculated session information to the model.

The prompt explicitly instructs the model to use only the supplied session information, preserve the integrity score and risk level, avoid inventing events, and avoid describing an event when its count is zero. The requested report contains an integrity assessment, risk level, score, key observations, suspicious activity and conclusion.

8.2 Validation

The validation module checks whether the generated report contains the expected integrity score and risk label. It also checks whether tab-switch, focus-loss and face-absence information is consistent with the supplied event counts. The overall validation result is the logical combination of these checks.

8.3 Post-Submission Processing

When the candidate submits an active exam, the session is marked as submitted. If a no-face interval is still open at submission time, the application closes the interval using the submission timestamp. It then calculates the score, creates the AI report, and stores the result in session_integrity_reports.

Following submission, the candidate sees a confirmation screen showing the exam name, submission time and status, followed by a results page displaying the completed session's integrity score, risk level and AI-generated summary.

9. Testing, Limitations, Future Scope and Conclusion

9.1 Testing Approach

Test Area	Expected Result
Registration	Valid candidate is created; duplicate email is rejected.
Login	Correct credentials create a session; invalid credentials are rejected.
Photo validation	No-face and multi-face images are rejected; one-face image is accepted.
System check	Browser/camera readiness status is shown before declaration.
Face monitoring	Face status changes are detected and recorded during an active exam.
Browser events	Tab/focus events are stored against the active session.
Submission	Session changes to submitted and the report is generated.
Scoring	Score remains within 0–100 and maps to the configured risk thresholds.
AI validation	Report consistency checks return pass/fail results.
Analytics	Completed-session charts and event information are displayed.

9.2 Current Limitations

- Haar Cascade is a face detector; it does not provide face recognition or liveness verification.
- Browser monitoring is limited to the events implemented by the examination page.
- The integrity score is a rule-based project metric and should be interpreted as an analytical indicator, not as proof of misconduct.
- K-Means clustering is exploratory and depends on the selected session features and the available dataset.
- AI-generated text depends on the configured external model/API and is validated against supplied session fields.

9.3 Future Scope

- Add stronger liveness and identity-verification techniques when project requirements permit.
- Add configurable examination policies and administrator-managed thresholds.
- Provide richer evidence views and export options for completed sessions.
- Improve scalability by moving from local SQLite to a production database when required.
- Add automated end-to-end tests and performance monitoring for larger examination loads.
- Extend the administrative dashboard with filters, session search and trend summaries.

9.4 Conclusion

The Online Exam Monitoring & Integrity Analytics Platform integrates web development, computer vision, database management, data science and generative AI into a single examination workflow. Its main contribution is the structured collection of examination-session evidence followed by transparent scoring, analytical visualization and a validated AI-assisted summary. The modular design also leaves clear opportunities for future improvements in verification, scalability, administration and testing.